

مستندات پروژه docker-anywhere

کامل فارسی – عامل اجرای دستور از راه دور با ارتباط خروجی محور (Outbound-Only) برای مدیریت Docker و سرور تولید راهنمای

خروجی) و Control Server مرکزی (صف دستورات و تاریخچه). هیچ پورت ورودی برای اجرای دستور از اینترنت روی سرور تولید باز نیست. خلاصه: این پروژه دو بخش دارد – Agent روی سرور تولید (فقط درخواست

۱. معرفی کلی پروژه

Python، UV ساخته شده است. هدف: مدیریت امن Docker و سرور از راه دور بدون باز کردن API ورودی روی تولید. پروژه docker-anywhere با FastAPI

مشکل API ورودی: هر مهاجم می‌تواند به POST /execute حمله کند.

راه‌حل: Agent هر ۱۵ ثانیه Control Server را poll می‌کند، فقط دستورات Whitelist اجرا می‌شود، نتیجه برمی‌گردد.

دو پوشه اصلی

اجرا، گزارش نتیجه، poll: روی سرور تولید – agent/

سرور مرکزی: صف دستور، احراز هویت، تاریخچه – control_server/

۲. معماری و جریان کار

```
Operator --POST /api/v1/commands--> Control Server
Agent <--GET /api/v1/poll----- Control Server (every 15s)
Agent --POST /api/v1/results-----> Control Server
Agent: only GET /health on 127.0.0.1 (local monitoring)
```

۱. اپراتور دستور را با کلید Operator به صف می‌فرستد.

۲. بعدی دستور را می‌گیرد poll در Agent.

۳. executor.py (بدون argv دستور را به shell=True) تبدیل می‌کند.

۴. نتیجه stdout/stderr/returncode به Control Server برمی‌گردد.

۳. پوشه agent / – عامل روی سرور تولید

FastAPI نقطه ورود – agent/main.py

هیچ GET /: systemmd وضعیت عامل برای GET /health: poller توقف shutdown: poller تنظیم لاگ و شروع startup: اجرای دستور از راه دور وجود ندارد endpoint پاسخ ساده.

Pydantic تنظیمات – agent/config.py

مسیرهای LOG_FILE و Docker، ALLOWED_SCRIPTS_DIR، REBOOT_CONFIRMATION_TOKEN، timeout، agent/.env: AGENT_ID، API_KEY، HMAC_SECRET، CONTROL_SERVER_URL، POLL_INTERVAL_SECONDS خواندن

مدل‌های داده – agent/models.py

ActionType (enum مجاز)، CommandPayload، ExecutionResult، HealthResponse، CommandStatus.

امنیت – agent/security.py

هدرها: HMAC-SHA256 با signed_request و signed_headers. audit_event و setup_logging (لاگ JSON). X-API-Key، X-Timestamp، X-Nonce، X-Signature. RateLimiter برای /health.

اجرای دستور – agent/executor.py

فقط از: regex ثابت. اعتبارسنجی نام سرویس با argv به action هر map _build_command. confirmation_token نیاز به: server_reboot. path traversal ضد، ALLOWED_SCRIPTS_DIR. async و timeout. execute_command: نقطه ورود اصلی.

polling حلقه – agent/poller.py

AgentPoller: httpx AsyncClient، GET poll، execute، POST results. backoff نگرهاری در خطا. last_poll_at نمایی در خطا. health برای commands_executed.

agent/.env.example

مستندات docker-anywhere
نمونه متغیرهای محیط – کپی به env.

agent/agent.service

نمونه hardening امنیتی با systemd

agent/__init__.py

علامت‌گذاری پکیج Python

۴. پوشه `control_server/` – سرور کنترل

سرور مرکزی. داده‌ها فعلاً in-memory است – برای تولید واقعی PostgreSQL/Redis پیشنهاد می‌شود.

`control_server/main.py` – API

سلامت سرور – GET /health

Operator اپراتور: صف کردن دستور (کلید – POST /api/v1/commands)

pending برداشتن دستور: Agent – GET /api/v1/poll?agent_id=...

ارسال نتیجه اجرا: Agent – POST /api/v1/results

اپراتور: مشاهده تاریخچه – GET /api/v1/history

`control_server/config.py`

AGENT_API_KEY/HMAC (مشترک با Agent)، OPERATOR_API_KEY/HMAC، MAX_TIMESTAMP_SKEW، rate limit.

`control_server/security.py`

verify_signed_request: API key، timestamp، nonce، HMAC. ضد replay و timing attack.

`control_server/models.py`

agent.models. استفاده مجدد از PushCommandRequest، QueuedCommand، HistoryEntry

`control_server/cli.py`

دستور امضا شده بدون نوشتن کد push اپراتور برای CLI

`control_server/.env.example`

نمونه تنظیمات سرور کنترل

۵. فایل‌های ریشه

(fastapi، httpx) و وابستگی‌ها UV تعریف پروژه – pyproject.toml

agent/main.py (ریشه) – نسخه قدیمی آزمایشی؛ جایگزین main.py

Python (۳.۱۲+) نسخه – python-version

۶. دستورات مجاز (Whitelist)

- service ری‌استارت سرویس (نیاز به `docker_restart`)
- `docker_rebuild` — `build` و `up -d`
- `docker_logs` — لاگ (`params.tail` ۱۰۰ پیش‌فرض)
- `docker_status` — `docker compose ps`
- `docker_stop` / `docker_start` — توقف / شروع
- `server_reboot` (نیاز به `confirmation_token` ریبوت)
- `run_script` — اسکریپت از پوشه مجاز (`params.script_name`)

۷. امنیت

- فقط ارتباط خروجی از Agent — بدون پورت دستور ورودی
- فقط `/health 127.0.0.1`
- روی هر درخواست `API key + HMAC`
- `replay` ضد `timestamp` و `Nonce`
- ثابت `argv` با `subprocess` — `shell` بدون `Whitelist`
- کلیدها: `chmod 600` — `hex 32` — `openssl rand` روی `env`.
- در تولید `Control Server` اجباری برای `HTTPS`

۸. نحوه اجرا با UV

```
uv sync
cp control_server/.env.example control_server/.env
uv run uvicorn control_server.main:app --host 0.0.0.0 --port 8000
```

```
cp agent/.env.example agent/.env
uv run uvicorn agent.main:app --host 127.0.0.1 --port 8080
```

```
export OPERATOR_API_KEY=...
export OPERATOR_HMAC_SECRET=...
uv run python -m control_server.cli push \
  --agent-id prod-server-01 --action docker_restart --service web
```

۹. افزودن دستور جدید

۱. مقدار جدید به `ActionType` در `agent/models.py`
۲. شاخه در `build_command()` داخل `agent/executor.py` — فقط `argv` ثابت
۳. ارسال از `CLI` یا `API` — هرگز رشته `shell` خام قبول نکنید

هشدار: هر action جدید باید ورودی validate شود و به دستور از پیش تعریف شده map شود.